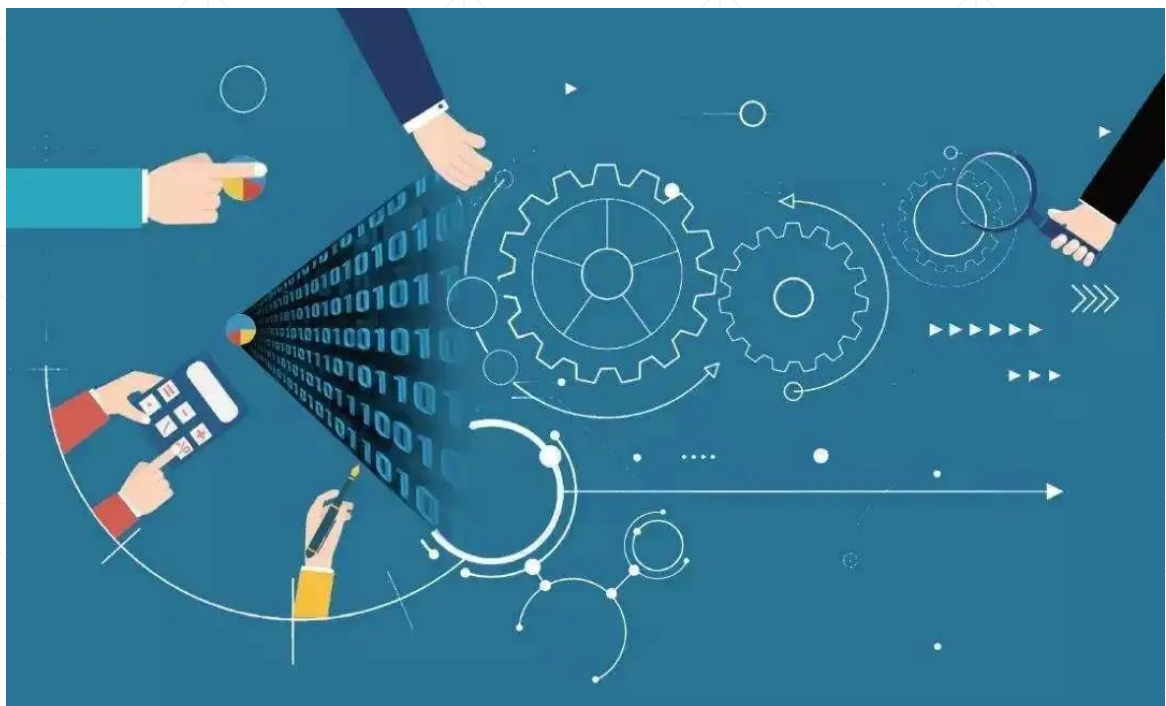




线程池任务的同步

北京理工大学计算机学院
金旭亮

概述



通常情况下，我们并不是简单地将任务提交给线程池去执行就完事了，这些任务之间可能有着“前赴后继”的关系，或者是我们可能要在任务完成之前，取消任务的执行。



本讲介绍一些基础的线程池任务同步编程技巧。

任务的取消

Future接口提供了cancel()方法用于取消任务:

```
boolean cancel(boolean mayInterruptIfRunning);
```

cancel方法的参数值为true时, 表示如果线程正在运行, 则向其发出中断请求, 这样, 线程可以主动中断任务的执行。这就要求线程执行的计算任务中要有相应的代码去响应外部的中断请求。

任务取消示例-1

```
//一个可以被取消的任务
1 usage
class CancelableTask implements Callable<String> {
    @Override
    public String call() throws Exception {
        while (true) {
            System.out.println("任务正在执行...");
            //JDK中的Thread.sleep方法, 是支持“提前中断休眠”的
            try {
                Thread.sleep( millis: 400);
            } catch (InterruptedException e) {
                throw new RuntimeException(e);
            }
        }
    }
}
```

一个支持中途取消的任务，必须有支持中途取消的代码，本例中直接使用了JDK中`Thread.sleep()`。

自己写，可以调用线程对象的`isInterrupted()`方法检测外界是否发出了取消请求，并且依此进行相应的处理，后面会有相应例子。

任务取消示例-2

示例在执行任务时，主线程等待2秒，然后向正在执行的任务发出“取消”请求。

```
Run: CancelTask x
"C:\Program Files\Java\jdk-17.0.3.
主线程提交任务给线程池
任务正在执行...
任务正在执行...
任务正在执行...
任务正在执行...
任务正在执行...
主线程发出取消任务的请求
Main: Cancelled: true
Main: Done: true
主线程退出

Process finished with exit code 0
```

```
public class CancelTask {
    public static void main(String[] args) {
        var executor : ExecutorService = Executors.newFixedThreadPool( nThreads: 2);
        var task = new CancelableTask();
        System.out.println("主线程提交任务给线程池");
        var result : Future<String> = executor.submit(task);
        //主线程休眠两秒，让线程池执行一些计算任务
        try {
            TimeUnit.SECONDS.sleep( timeout: 2);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println("主线程发出取消任务的请求");
        result.cancel( mayInterruptIfRunning: true);
        // 检查Future对象的状态
        System.out.printf("Main: Cancelled: %s\n", result.isCancelled());
        System.out.printf("Main: Done: %s\n", result.isDone());
        // 关闭线程池
        executor.shutdown();
        System.out.println("主线程退出");
    }
}
```

自定义支持中途取消的任务

需要在线程函数中添加中途取消的代码:

```
class CancelableTask2 implements Callable<String> {  
    @Override  
    public String call() throws Exception {  
        System.out.print("任务正在执行");  
        while (true) {  
            if (Thread.currentThread().isInterrupted()) {  
                System.out.println("响应外部取消请求, 跳出循环");  
                break;  
            }  
        }  
        return "任务结束";  
    }  
}
```



```
Run: CancelTask x  
"C:\Program Files\Java\jdk-17.0.3.1\bin"  
主线程提交任务给线程池  
任务正在执行主线程发出取消任务的请求  
响应外部取消请求, 跳出循环  
Main: Cancelled: true  
Main: Done: true  
主线程退出  
  
Process finished with exit code 0
```

多任务同步



如果有多个并发执行的任务，其中只要有一个完成了就行，一个典型场景就是使用多个线程同时在多个文件夹中搜索特定文件，只要找到了，就没必要等待其他的线程也工作结束了。



可以使用ExecutorService所定义的invokeAny()方法：

```
<T> T invokeAny(Collection<? extends Callable<T>> tasks)
```



此方法的返回值，就是最先完成的那个线程给出的任务结果。

多任务同步示例-1

//将要执行的任务，执行时间是随机的

1 usage

```
static String task() {  
    var threadName :String = Thread.currentThread().getName();  
    int ranValue = new Random().nextInt( origin: 1000, bound: 5000);  
    for (int i = 0; i < 4; i++) {  
        System.out.println(threadName + "正在处理" + i);  
        try {  
            Thread.sleep(ranValue);  
        } catch (InterruptedException e) {  
            throw new RuntimeException(e);  
        }  
    }  
    return threadName + "处理结束";  
}
```


多任务同步示例-2

```
public static void main(String[] args) {  
    ExecutorService executor = Executors.newCachedThreadPool();  
    //创建3个任务, 加入到集合中  
    List<Callable<String>> tasks = new ArrayList<>();  
    for (int i = 0; i < 3; i++) {  
        tasks.add(UseInvokeAny::task);  
    }  
    try {  
        //提交给线程池执行, 只要有一个运行结束, 此方法就返回  
        var result : String = executor.invokeAny(tasks);  
        System.out.println(result);  
    } catch (InterruptedException | ExecutionException e) {  
        throw new RuntimeException(e);  
    }  
    executor.shutdown();  
    System.out.println("程序退出");  
}
```

Run: UseInvokeAny x

"C:\Program Files\Java\jdk-17.0.3.1\
pool-1-thread-2正在处理0
pool-1-thread-1正在处理0
pool-1-thread-3正在处理0
pool-1-thread-2正在处理1
pool-1-thread-2正在处理2
pool-1-thread-1正在处理1
pool-1-thread-2正在处理3
pool-1-thread-3正在处理1
pool-1-thread-2处理结束
程序退出

Process finished with exit code 0

等待所有任务的结束



如果有多个任务投递给线程池执行，并且调用者希望等到它们全部运行结束，这时，可以使用`ExecutorService`所定义的`invokeAll()`方法达到目的：

```
<T> List<Future<T>> invokeAll(Collection<? extends Callable<T>> tasks)
```



这些任务的结果，放到一个集合中返回调用。

等待所有任务结束示例

示例中所定义的任务，休眠特定的时间之后，返回一个Result对象作为结果。

```
class Result {  
    2 usages  
    private String name;  
    2 usages  
    private int value;  
  
    "getter and setter"  
}
```

```
class InvokeTask implements Callable<Result> {  
    5 usages  
    private String name;  
    1 usage  
    public InvokeTask(String name) { this.name = name; }  
    @Override  
    public Result call() throws Exception {  
        System.out.printf("%s: 开始执行\n", this.name);  
        try {  
            Long duration = (long) (Math.random() * 10);  
            System.out.printf("%s: 等待 %d 秒出结果.\n", this.name, duration);  
            TimeUnit.SECONDS.sleep(duration);  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
        int value = 0;  
        for (int i = 0; i < 5; i++) {  
            value += (int) (Math.random() * 100);  
        }  
        Result result = new Result();  
        result.setName(this.name);  
        result.setValue(value);  
        System.out.printf("%s: 结束工作\n", this.name);  
        return result;  
    }  
}
```

```

public class InvokeAllDemo {
    public static void main(String[] args) {
        var executor : ExecutorService = Executors.newCachedThreadPool();
        List<InvokeTask> taskList = new ArrayList<>();
        for (int i = 0; i < 3; i++) {
            InvokeTask task = new InvokeTask( name: "Task-" + i);
            taskList.add(task);
        }
        List<Future<Result>> resultList = null;
        try {
            resultList = executor.invokeAll(taskList);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        executor.shutdown();
        System.out.printf("\n主线程打印结果\n");
        for (int i = 0; i < resultList.size(); i++) {
            Future<Result> future = resultList.get(i);
            try {
                Result result = future.get();
                System.out.printf("%s: %s\n", result.getName(), result.getValue());
            } catch (InterruptedException | ExecutionException e) {
                e.printStackTrace();
            }
        }
        System.out.println("\n所有的任务都结束了!");
    }
}

```

依次提取所有结果

示例启动了三个并行任务，只有等到三个任务都结束了，程序才输出结果。

```

Run: InvokeAllDemo x
"C:\Program Files\Java\jdk-17.0.3.1
Task-0: 开始执行
Task-1: 开始执行
Task-2: 开始执行
Task-0: 等待 9 秒出结果。
Task-1: 等待 3 秒出结果。
Task-2: 等待 3 秒出结果。
Task-1: 结束工作
Task-2: 结束工作
Task-0: 结束工作

主线程打印结果
Task-0: 267
Task-1: 359
Task-2: 232

所有的任务都结束了!

Process finished with exit code 0

```

小结



本讲主要介绍了一些最基本的线程池任务同步手段：

- (1) 提前取消一个任务的执行
- (2) 等待多个任务中的某个完成工作
- (3) 等待多个任务全部完成



必须指出，如果任务之间存在着复杂的同步关系，那么，仅使用上述手段是比较受限的，这时，可以采用JDK所提供的各种线程同步组件来实现，但这样一来，问题就变得复杂了。



更推荐的方式，是使用CompletableFuture所提供的Java异步编程方式，或者是使用RxJava这样的反应式编程框架，能比较好地解决任务同步的问题。